

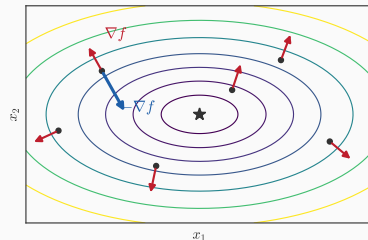
Optimization

3.3.1 Steepest descent method

José Firmino Aguilar Madeira

If we only know $\nabla f(\mathbf{x}_k)$, where should we go?

3.3 ended with this question. From 3.1: the gradient is normal to the level curve and points in the direction of steepest local **ascent**.

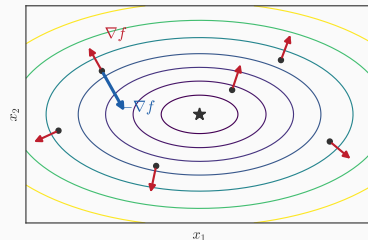


The figure from 3.1: ∇f (red) and $-\nabla f$ (blue).

If we only know $\nabla f(\mathbf{x}_k)$, where should we go?

3.3 ended with this question. From 3.1: the gradient is normal to the level curve and points in the direction of steepest local **ascent**.

Among directions of the same length, $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), with equality at $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: the **locally steepest** descent.



The figure from 3.1: ∇f (red) and $-\nabla f$ (blue).

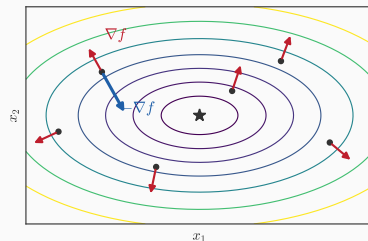
If we only know $\nabla f(\mathbf{x}_k)$, where should we go?

3.3 ended with this question. From 3.1: the gradient is normal to the level curve and points in the direction of steepest local **ascent**.

Among directions of the same length, $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), with equality at $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: the **locally steepest** descent.

Mind the name

Steepest descent = the steepest direction **at the point**, for fixed $\|\mathbf{d}\|$ — not “the algorithm that reaches the minimum fastest”.



The figure from 3.1: ∇f (red) and $-\nabla f$ (blue).

If we only know $\nabla f(\mathbf{x}_k)$, where should we go?

3.3 ended with this question. From 3.1: the gradient is normal to the level curve and points in the direction of steepest local **ascent**.

Among directions of the same length, $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), with equality at $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: the **locally steepest** descent.

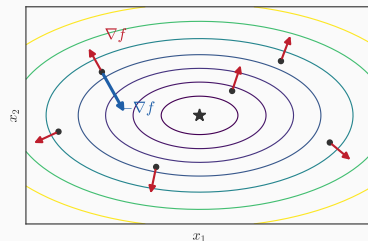
Mind the name

Steepest descent = the steepest direction **at the point**, for fixed $\|\mathbf{d}\|$ — not “the algorithm that reaches the minimum fastest”.

And **how far to go**? That is a separate decision: the step α_k , by the line search of Ch. 2.

3.3 chooses $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$

Ch. 2 chooses α_k

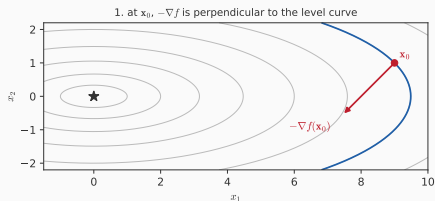


The figure from 3.1: ∇f (red) and $-\nabla f$ (blue).

One iteration by hand — and orthogonality appears

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, minimum at $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

1. **Direction:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.



One iteration by hand — and orthogonality appears

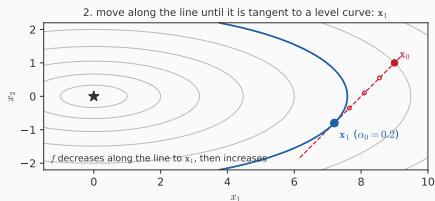
$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, minimum at $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

1. **Direction:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

2. **How far to go:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha\mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0.2.$$



One iteration by hand — and orthogonality appears

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, minimum at $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

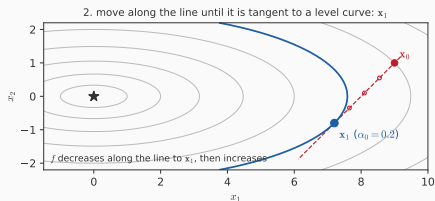
1. **Direction:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

2. **How far to go:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0.2.$$

3. **New point:** $\mathbf{x}_1 = \mathbf{x}_0 + 0.2 \mathbf{d}_0 = (7.2, -0.8)$, $f = 28.8$ (it was 45).



One iteration by hand — and orthogonality appears

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, minimum at $(0,0)$;
 $\mathbf{x}_0 = (9, 1)$.

1. **Direction:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

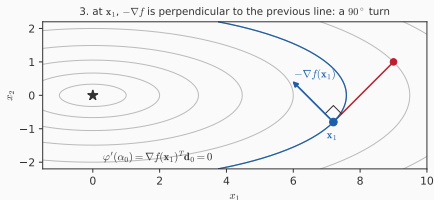
2. **How far to go:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0.2.$$

3. **New point:** $\mathbf{x}_1 = \mathbf{x}_0 + 0.2 \mathbf{d}_0 = (7.2, -0.8)$, $f = 28.8$ (it was 45).

4. **Surprise:** $\mathbf{g}_1 = (7.2, -7.2)$ and
 $\nabla f(\mathbf{x}_1)^T \mathbf{d}_0 = -64.8 + 64.8 = 0$.



One iteration by hand — and orthogonality appears

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, minimum at $(0,0)$;
 $\mathbf{x}_0 = (9,1)$.

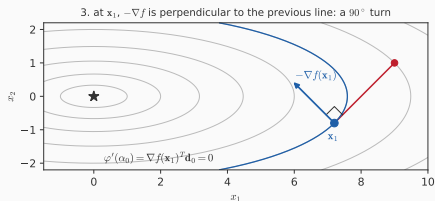
1. **Direction:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9,9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9,-9)$.

2. **How far to go:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$
$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0.2.$$

3. **New point:** $\mathbf{x}_1 = \mathbf{x}_0 + 0.2 \mathbf{d}_0 = (7.2, -0.8)$, $f = 28.8$ (it was 45).

4. **Surprise:** $\mathbf{g}_1 = (7.2, -7.2)$ and
 $\nabla f(\mathbf{x}_1)^T \mathbf{d}_0 = -64.8 + 64.8 = 0$.



Why: exact line search

At the minimum of φ : $\varphi'(\alpha_k) = \mathbf{g}_{k+1}^T \mathbf{d}_k = 0$. Since

$$\mathbf{d}_k = -\mathbf{g}_k: \quad \boxed{\mathbf{g}_{k+1}^T \mathbf{g}_k = 0}$$

This holds for any differentiable f , provided α_k minimizes φ (it is not specific to quadratics): **consecutive gradients are orthogonal**. In 3.3.3 we will look for a better notion — conjugacy with respect to $H = \nabla^2 f$.

The algorithm

Steepest descent method: $\mathbf{x}_0, \varepsilon_g, \varepsilon_x, k_{\max}$

```
1:  $k \leftarrow 0$ 
2: loop
3:    $\mathbf{g}_k \leftarrow \nabla f(\mathbf{x}_k)$ 
4:   if  $\|\mathbf{g}_k\| \leq \varepsilon_g$  or  $k \geq k_{\max}$  then return  $\mathbf{x}_k$ 
5:   end if
6:    $\alpha_k \leftarrow \arg \min_{\alpha > 0} f(\mathbf{x}_k - \alpha \mathbf{g}_k)$  ▷ line search
7:    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha_k \mathbf{g}_k$ 
8:   if  $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| / \max(1, \|\mathbf{x}_k\|) \leq \varepsilon_x$  then return  $\mathbf{x}_{k+1}$ 
9:   end if
10:   $k \leftarrow k + 1$ 
11: end loop
```

Notation (see 2.1): with $\varphi(\alpha) = f(\mathbf{x}_k - \alpha \mathbf{g}_k)$, $\min_{\alpha > 0} \varphi(\alpha)$ is the *minimum value*; $\arg \min_{\alpha > 0} \varphi(\alpha)$ is the α **at which it is attained** (the minimizer).

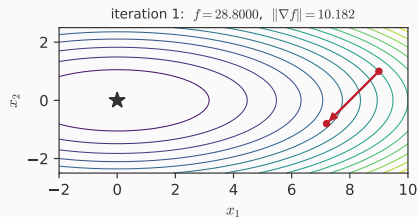
gradient $\rightarrow$ direction $-\mathbf{g}_k$
 $\rightarrow$ line search (α_k) $\rightarrow$ new point
 $\rightarrow$ stop?

- **Cost per iteration:** $n_g = 1$ ($2n$ evaluations of f if the gradient is numerical) + the evaluations of f in the line search.
- **Memory:** only $\mathbf{x}_k$ and $\mathbf{g}_k$ — $O(n)$.
- **Exact search:** $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$. **In practice** an *inexact* line search is normally used, which only requires sufficient decrease (for example, Armijo/Wolfe conditions).

The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

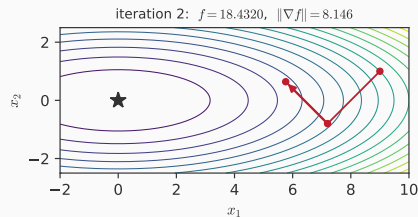
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

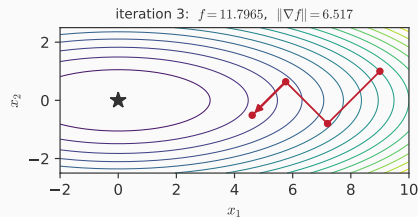
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

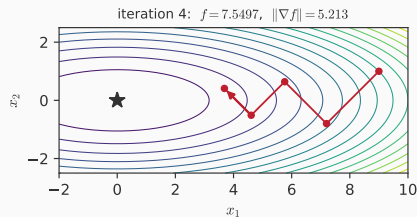
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

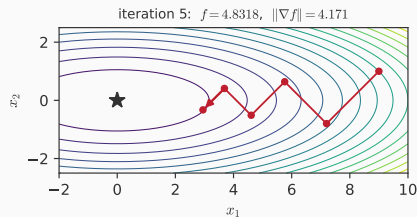
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000
4	(3.6864, 0.4096)	7.54975	5.2134	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

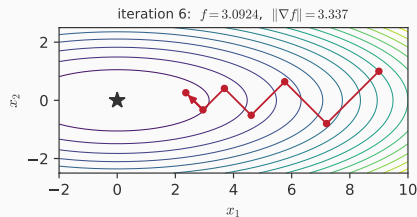
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000
4	(3.6864, 0.4096)	7.54975	5.2134	0.2000
5	(2.9491, -0.3277)	4.83184	4.1707	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

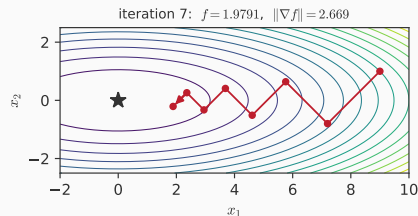
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000
4	(3.6864, 0.4096)	7.54975	5.2134	0.2000
5	(2.9491, -0.3277)	4.83184	4.1707	0.2000
6	(2.3593, 0.2621)	3.09238	3.3365	0.2000



The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000
4	(3.6864, 0.4096)	7.54975	5.2134	0.2000
5	(2.9491, -0.3277)	4.83184	4.1707	0.2000
6	(2.3593, 0.2621)	3.09238	3.3365	0.2000
7	(1.8874, -0.2097)	1.97912	2.6692	0.2000

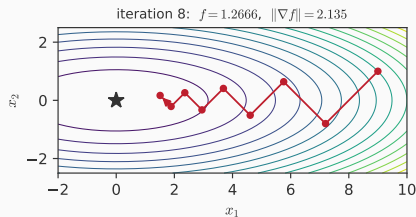


The zigzag path

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ from $(9, 1)$, exact line search.

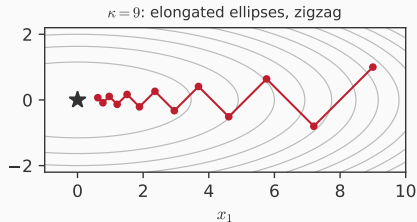
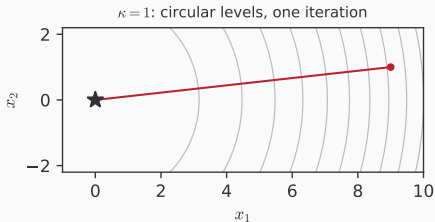
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9.0000, 1.0000)	45.00000	12.7279	—
1	(7.2000, -0.8000)	28.80000	10.1823	0.2000
2	(5.7600, 0.6400)	18.43200	8.1459	0.2000
3	(4.6080, -0.5120)	11.79648	6.5167	0.2000
4	(3.6864, 0.4096)	7.54975	5.2134	0.2000
5	(2.9491, -0.3277)	4.83184	4.1707	0.2000
6	(2.3593, 0.2621)	3.09238	3.3365	0.2000
7	(1.8874, -0.2097)	1.97912	2.6692	0.2000
8	(1.5099, 0.1678)	1.26664	2.1354	0.2000

Each iteration multiplies f by 0.64 and the step is always $\alpha = 0.2$: the path is geometrically similar from one iteration to the next. It takes **74 iterations** to reach $\|\nabla f\| < 10^{-6}$ — on a quadratic in two variables.



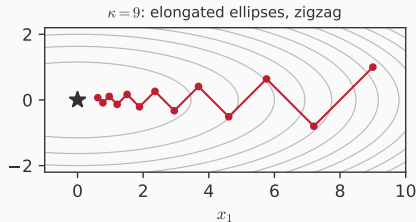
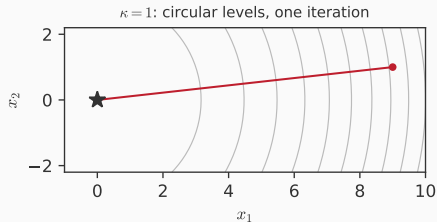
The shape of the level curves decides

The steps are always perpendicular; what decides whether this is costly is the **shape** of the level curves.



The shape of the level curves decides

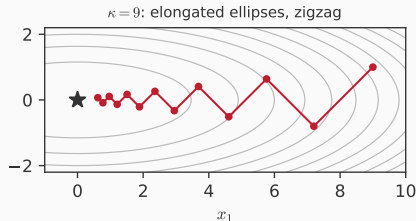
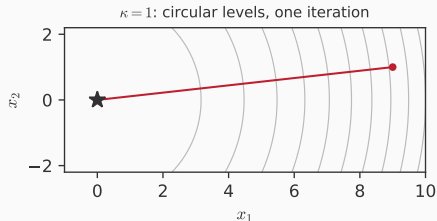
The steps are always perpendicular; what decides whether this is costly is the **shape** of the level curves.



Why, with circles, is one iteration enough?

The shape of the level curves decides

The steps are always perpendicular; what decides whether this is costly is the **shape** of the level curves.



Why, with circles, is one iteration enough? With circles ($\kappa = 1$), the normal to the level curve passes through the centre: $-\nabla f$ points **to the minimum**. With 3 : 1 ellipses ($\kappa = \lambda_{\max}/\lambda_{\min}$ of the Hessian: here 9/1; see next page), it points to the opposite “wall”, and the 90° steps go from wall to wall: 74 iterations.

Conditioning: why elongation = slowness

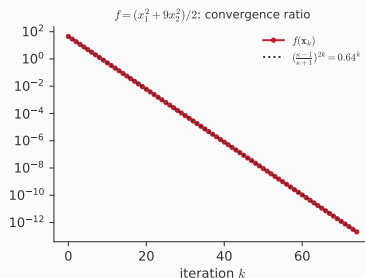
Assumptions: $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ with A **symmetric positive definite**; steepest descent with **exact** line search. Let $\mathbf{x}^*$ be the minimizer and $E(\mathbf{x}) = f(\mathbf{x}) - f^* = \frac{1}{2} \|\mathbf{x} - \mathbf{x}^*\|_A^2$, with $\|\mathbf{v}\|_A^2 = \mathbf{v}^T A \mathbf{v}$.

Bound (Kantorovich)

$$E(\mathbf{x}_{k+1}) \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 E(\mathbf{x}_k), \quad \kappa = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)},$$

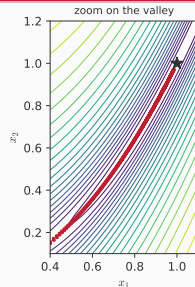
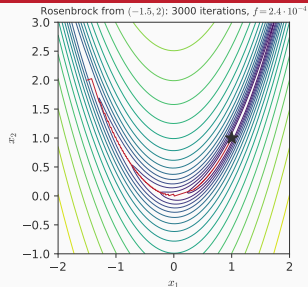
that is, $\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_A \leq \frac{\kappa-1}{\kappa+1} \|\mathbf{x}_k - \mathbf{x}^*\|_A$: factor $\left(\frac{\kappa-1}{\kappa+1} \right)^2$ on the error in f , factor $\frac{\kappa-1}{\kappa+1}$ on the error in the A -norm.

Here $\kappa = 9$, $f^* = 0$: the bound gives 0.64 for f , attained at every iteration from $(9, 1)$. $\kappa = 1$: one iteration; $\kappa = 100$: 0.96 — hundreds of iterations. **Linear** convergence, slow if κ is large.



It is a worst-case bound; for other quadratics there may be no equality at each iteration. The semi-axes of the level ellipses are in the ratio $\sqrt{\kappa}$.

Rosenbrock: reaching the valley is not the same as travelling along it



- It reaches the valley in a few iterations (the gradient is large and points there) and then **crawls**: after 3000 iterations still $f = 2.4 \cdot 10^{-4}$, $\mathbf{x} \approx (0.98, 0.97)$.
- At the minimum $(1, 1)$ the Hessian has $\kappa \approx 2.5 \cdot 10^3$: with exact line search, consecutive gradients are orthogonal and progress along the valley is very slow.
- Even so, it is one gradient and one line search per iteration: with very large n , first-order methods become attractive for their low cost and memory — it is the starting point of neural network training (“gradient descent”).

The core of one iteration, in MATLAB:

```
g = grad(x);           %  $n_g = 1$ 
d = -g;                % direction
phi = @(a) f(x + a*d); % line search
alpha = fminbnd(phi, 0, amax); % Ch. 2
% (if  $\alpha \sim \text{amax}$ , enlarge  $\text{amax}$ )
x = x + alpha*d;
```

fminbnd uses golden section + parabolic interpolation (2.2–2.3).

- In the example ($\varepsilon_g = 10^{-6}$): 74 iterations, $n_g = 75$, about 6 evaluations of f per line search with this fminbnd (with the high-precision search of the figures, about 24).
- Without an analytical gradient, each gradient by central differences costs $n_f = 2n$ (2.6).
- The full functions (SteepestDescent, with info.nfev and info.ngev; GradFD) are in the course companion code.

The price of the steepest descent method

Advantages

- needs only ∇f (first order);
- cheap per iteration; memory $O(n)$;
- simple; it is the basis of many methods for very large n .

Limitations

- zigzag: consecutive gradients are orthogonal;
- linear convergence, very slow if κ is large;
- ignores curvature: in narrow valleys it points at the wall, not at the bottom.

“Global convergence” of a local algorithm

Global = from *any* starting point (under the assumptions: bounded level set, Lipschitz gradient (R), suitable line search) one obtains approximate *stationarity* ($\|\nabla f(\mathbf{x}_k)\| \rightarrow 0$, or at least $\liminf \|\nabla f(\mathbf{x}_k)\| = 0$, depending on the assumptions): the guarantee is stationarity, not necessarily convergence of the iterates to a single point. **It does not mean convergence to the global minimum** — in Ch. 5, “global” will refer to the optimum.

KEY TAKEAWAYS — 3.3.1

1. With only ∇f : $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, the **locally steepest** descent; the step α_k comes from the line search (Ch. 2).
2. With exact line search, $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$: consecutive gradients are **orthogonal** — the zigzag.
3. On SPD quadratics, with exact line search, the error in f satisfies $E_{k+1} \leq \left(\frac{\kappa-1}{\kappa+1}\right)^2 E_k$: **linear** convergence, which degrades as κ grows.
4. Low cost per iteration: one gradient and one line search; memory $O(n)$.
5. “Global convergence” = stationarity from any $\mathbf{x}_0$, under assumptions — **not** the global minimum.

Steepest descent uses only the **slope** and suffers from the geometry. Two questions:

Can we use the **curvature**? → Newton (3.3.2)

Can we use **memory** without paying for the Hessian? → conjugate gradients (3.3.3)

References Full list: *Course bibliography*, at the end of the course PDF.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Ch. 3.
- Fletcher, R., *Practical Methods of Optimization*, 2nd ed., Wiley, 1987.

Resources

- Companion code: SteepestDescent (`info.nfev`, `info.ngev`), GradFD. MATLAB: `fminbnd` (line search).